Transient Analysis of an RC Circuit

Problem 1.1.16 — EC 2007

EE25BTECH11016- CH. Taraka Abhinav

Problem 1.1.16 – EC 2007

In the circuit shown, $V_C = 0$ V at $t = 0$ sec. For $t > 0$, the capacitor current $i_c(t)$ (t in seconds) is:

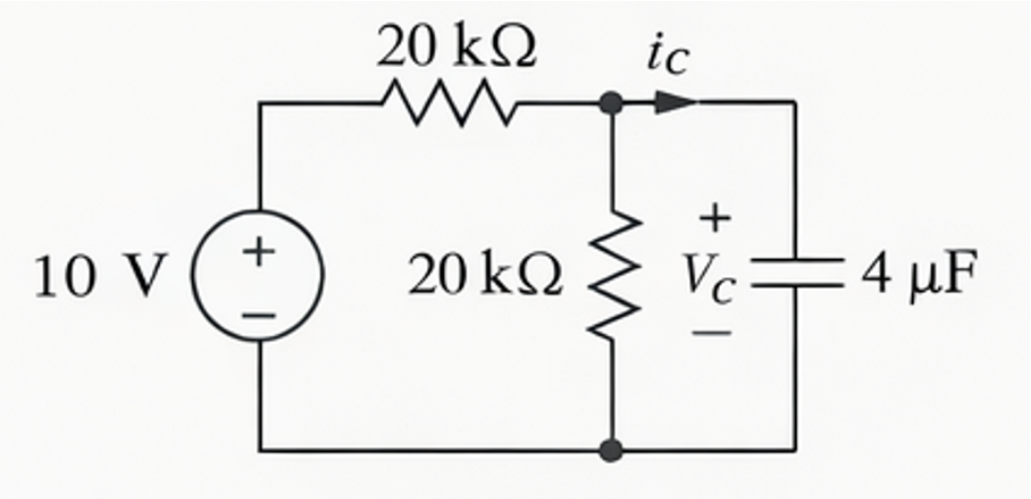
Options:

a. $0.50\, e^{-25t}$ mA

b. $0.25\, e^{-25t}$ mA

c. $0.50\, e^{-12.5t}$ mA

d. $0.25\, e^{-6.25t}$ mA



Initial Condition

Given: $V_C(t = 0^-) = 0\,\mathrm{V}$

A capacitor **cannot change its voltage instantaneously** because an instantaneous change requires **infinite current**, which is physically impossible.

Therefore:

$$V_C(t = 0^+) \;=\; V_C(t = 0^-) \;=\; \mathbf{0\,V}$$

At $t = 0^+$, since $V_C = 0$, the capacitor behaves as a **short circuit**.

Steady-State Voltage V_∞

As $t \to \infty$, the capacitor is **fully charged**

$\Rightarrow$ no current flows into it $\Rightarrow$ capacitor is an **open circuit**

The two resistors then form a simple **Voltage Divider**:

$$V_\infty = V_s \times \frac{R_2}{R_1 + R_2} = 10 \times \frac{20\,\mathrm{k}}{20\,\mathrm{k} + 20\,\mathrm{k}} = 10 \times \frac{1}{2}$$

$$V_\infty = 5\,\mathrm{V}$$

Thévenin Resistance R_{th}

Deactivate the independent voltage source
(replace with a **short circuit**)

Looking from the capacitor terminals, both $20\,\mathrm{k}\Omega$ resistors are now connected **in parallel**:

$$R_{th} = 20\,\mathrm{k} \parallel 20\,\mathrm{k} = \frac{20 \times 20}{20 + 20}\,\mathrm{k} = \frac{400}{40}\,\mathrm{k}$$

$$R_{th} = 10\,\mathrm{k}\Omega$$

Time Constant τ

The time constant for an RC circuit is:

$$\tau = R_{th} \cdot C$$

Substituting $R_{th} = 10\,\mathrm{k}\Omega$ and $C = 4\,\mu\mathrm{F}$:

$$\tau = 10 \times 10^3 \times 4 \times 10^{-6} = 40 \times 10^{-3}\,\mathrm{s}$$

$$\tau = 40\,\mathrm{ms}$$

Therefore the decay exponent:

$$\frac{1}{\tau} = \frac{1}{0.04} = \mathbf{25\,s^{-1}}$$

Capacitor Voltage $V_C(t)$

The standard RC complete response formula:

$$V_C(t) \;=\; V_\infty \;+\; \left[V_C(t=0^+) - V_\infty\right] e^{-t/\tau}$$

Substituting $V_\infty = 5\,\mathrm{V}$, $\quad V_C(t=0^+) = 0\,\mathrm{V}$, $\quad 1/\tau = 25$:

$$V_C(t) \;=\; 5 \;+\; [0-5]\, e^{-25t}$$

$$V_C(t) \;=\; 5\left(1 - e^{-25t}\right)\mathrm{V}$$

Check: At $t = 0$: $V_C = 5(1-1) = 0\,\mathrm{V}$ ✓ As $t \to \infty$: $V_C \to 5\,\mathrm{V}$ ✓

Capacitor Current $i_c(t)$

Using the fundamental capacitor relation:

$$i_c(t) \;=\; C\,\frac{dV_C}{dt}$$

Differentiating $V_C(t) = 5(1 - e^{-25t})$:

$$\frac{dV_C}{dt} \;=\; 5 \times 25\,e^{-25t} \;=\; 125\,e^{-25t}\,\mathrm{V/s}$$

Multiplying by $C = 4 \times 10^{-6}\,\mathrm{F}$:

$$i_c(t) \;=\; 4 \times 10^{-6} \;\times\; 125\,e^{-25t} \;=\; 500 \times 10^{-6}\,e^{-25t}\,\mathrm{A}$$

$$i_c(t) \;=\; 0.50\,e^{-25t}\,\mathrm{mA}$$

Answer: Option (a)

Verification via KCL

At the junction node, applying KCL:

$$i_c = \underbrace{\frac{10 - V_C}{20\,\mathrm{k}}}_{i_1 \text{ (series R)}} - \underbrace{\frac{V_C}{20\,\mathrm{k}}}_{i_2 \text{ (shunt R)}} = \frac{10 - 2V_C}{20\,\mathrm{k}}$$

Substituting $V_C = 5(1 - e^{-25t})$, so $2V_C = 10(1 - e^{-25t})$:

$$i_c(t) = \frac{10 - 10(1 - e^{-25t})}{20\,\mathrm{k}} = \frac{10\,e^{-25t}}{20\,\mathrm{k}}$$

$$i_c(t) = 0.50\,e^{-25t}\,\mathrm{mA} \checkmark$$

Advanced: ODE Formulation

Applying **KCL** at the capacitor node for $t > 0$:

$$C\frac{dV_C(t)}{dt} + \frac{V_C(t)}{R_2} = \frac{V_s - V_C(t)}{R_1}$$

Rearranging into the standard first-order linear ODE form:

$$\frac{dV_C(t)}{dt} + \left(\frac{R_1 + R_2}{R_1 R_2 C}\right) V_C(t) = \frac{V_s}{R_1 C}$$

Substituting circuit values:

$$\frac{dV_C(t)}{dt} + \mathbf{25\,V_C(t)} = \mathbf{125}$$

Advanced: ODE Solution

From the derived ODE:

$$\frac{dV_C(t)}{dt} + 25\,V_C(t) = 125$$

The complete response is the sum of **natural** and **forced** responses:

$$V_C(t) = Ke^{-25t} + 5$$

Applying the initial condition $V_C(0^+) = 0$, we find $K = -5$.

$$V_C(t) = 5\left(1 - e^{-25t}\right)\,\mathrm{V}$$

Advanced: Transfer Function $H(s)$

Analyzing the circuit as a system mapping $V_s(t) \to V_C(t)$ in the s-domain.

The Transfer Function is found via voltage division with impedances:

$$H(s) = \frac{V_C(s)}{V_s(s)} = \frac{R_2 \parallel \frac{1}{sC}}{R_1 + \left(R_2 \parallel \frac{1}{sC}\right)}$$

Simplifying and substituting values yields:

$$H(s) = \frac{12.5}{s + \mathbf{25}}$$

The pole is located at $s = -25\,\mathrm{rad/s}$, *which defines the system's cutoff frequency.*

Advanced: Laplace Transform

In the s-**domain**, the input is a step: $V_s(s) = \dfrac{10}{s}$.

Using Nodal Analysis and solving for $V_C(s)$:

$$V_C(s) = \frac{125}{s(s+25)}$$

Applying **Partial Fraction Expansion**:

$$V_C(s) = \frac{5}{s} - \frac{5}{s+25}$$

$$V_C(t) = 5\left(1 - e^{-25t}\right) u(t) \text{ V}$$

Advanced: State-Space Formulation

For modern control theory, we express the circuit as:

$$\dot{\mathbf{x}}(\mathbf{t}) \;=\; \mathbf{A}\mathbf{x}(\mathbf{t}) + \mathbf{B}\mathbf{u}(\mathbf{t})$$

Let the state variable be $x(t) = V_C(t)$ and the input $u(t) = V_s(t)$.

From the earlier ODE, we directly extract the matrices:

$$\dot{\mathbf{x}}(\mathbf{t}) \;=\; \underbrace{-25}_{\mathbf{A}}\,\mathbf{x}(\mathbf{t}) \;+\; \underbrace{12.5}_{\mathbf{B}}\,\mathbf{u}(\mathbf{t})$$

The eigenvalue of $\mathbf{A}$ *is* $\lambda = -25$.
Since $\lambda < 0$, *the system is* **strictly stable**.

Advanced: Energy Dynamics

Steady-State Energy Stored ($t \to \infty$):

$$E_C = \frac{1}{2}CV_\infty^2 = \frac{1}{2}(4\,\mu\mathrm{F})(5\,\mathrm{V})^2 = \mathbf{50}\,\mu\mathrm{J}$$

Instantaneous Power Delivered:

$$p_C(t) = V_C(t) \cdot i_C(t)$$

$$p_C(t) = 2.5\left(e^{-25t} - e^{-50t}\right)\,\mathrm{mW}$$

Maximum power transfer occurs at $t = \dfrac{\ln 2}{25} \approx \mathbf{27.7}\,\mathrm{ms}$.

Numerical Verification: RK4 Method (1/3)

Why Runge-Kutta 4 (RK4)?

Analytical solutions are not always feasible for complex circuits. Numerical methods like RK4 provide highly accurate approximations with a global error of $\mathcal{O}(h^4)$.

The governing **Ordinary Differential Equation (ODE)** for our circuit is:

$$\frac{dV_C}{dt} = f(t, V_C) = 125 - 25\,V_C$$

We evaluate the voltage V_C at discrete time steps t_n, such that:

$$t_{n+1} = t_n + h$$

where h is the chosen **time step**.

Numerical Verification: RK4 Method (2/3)

RK4 calculates four different **slopes** (k_1, k_2, k_3, k_4) across the interval $[t_n, t_{n+1}]$ to find a highly accurate weighted average.

$$k_1 = f(t_n,\ V_n)$$

(Slope at the beginning of the interval)

$$k_2 = f\left(t_n + \frac{h}{2},\ V_n + \frac{h}{2}k_1\right)$$

(Slope at the midpoint, estimated using k_1)

$$k_3 = f\left(t_n + \frac{h}{2},\ V_n + \frac{h}{2}k_2\right)$$

(Slope at the midpoint, refined using k_2)

$$k_4 = f(t_n + h,\ V_n + h\,k_3)$$

(Slope at the end of the interval, estimated using k_3)

Numerical Verification: RK4 Method (3/3)

The Final Update Rule: The next voltage value V_{n+1} is computed by adding a weighted average of the four slopes to the current value V_n:

$$V_{n+1} = V_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Implementation Details for our Circuit:

- **Initial Condition:** $t_0 = 0,\ V_C(0) = 0\,\mathrm{V}$
- **Step Size:** $h = 0.5 \times 10^{-3}\,\mathrm{s}\ (0.5\,\mathrm{ms})$
- **Simulation Time:** $t_{\text{end}} = 200\,\mathrm{ms}$

Greater weight is given to the midpoint slopes (k_2, k_3) as they better represent the function's average behavior over the given interval.

SPICE Simulation: Overview

NGSpice is used to simulate the exact same RC circuit and independently verify both $V_C(t)$ and $i_c(t)$.

Circuit Netlist Elements:

Element	Value	Nodes
V_s	$10\,\mathrm{V}$ DC	$1 \rightarrow$ GND
R_1	$20\,\mathrm{k}\Omega$	Node 1 $\rightarrow$ Node 2
R_2	$20\,\mathrm{k}\Omega$	Node 2 $\rightarrow$ GND
C	$4\,\mu\mathrm{F}$, IC $= 0$	Node 2 $\rightarrow$ GND

Transient analysis: step $= 0.5\,\mathrm{ms}$, stop $= 200\,\mathrm{ms}$

Node 2 voltage $= V_C(t)$; *current through* $C = i_c(t)$

RK4 Verification – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

#  Circuit parameters
Vs = 10.0 # Source voltage (V)
R1 = 20e3 # Series resistor (Ohm)
R2 = 20e3 # Shunt resistor (Ohm)
C = 4e-6 # Capacitance (F)
Rth = R1 * R2 / (R1 + R2) # Thevenin resistance
Vinf = Vs * R2 / (R1 + R2) # Steady-state voltage

# ODE: dVc/dt = f(Vc) = 125 - 25*Vc
def f(Vc):
   return (Vinf - Vc) / (Rth * C) # = 125 - 25*Vc

#  RK4 solver
h = 0.5e-3 # step size (0.5 ms)
t_end = 0.200 # end time (200 ms)
N = int(t_end / h)

t_rk = np.zeros(N + 1)
Vc_rk = np.zeros(N + 1)
```

RK4 Verification – Python Code II

```
Vc_rk[0] = 0.0 # initial condition

for n in range(N):
   k1 = f(Vc_rk[n])
   k2 = f(Vc_rk[n] + h/2 * k1)
   k3 = f(Vc_rk[n] + h/2 * k2)
   k4 = f(Vc_rk[n] + h * k3)
   Vc_rk[n+1] = Vc_rk[n] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
   t_rk[n+1] = t_rk[n] + h

#  Exact (theoretical) solution
t_ex = np.linspace(0, t_end, 2000)
Vc_ex = Vinf * (1 - np.exp(-25 * t_ex))
ic_ex = (Vinf / Rth) * np.exp(-25 * t_ex) * 1000 # mA
ic_rk = np.array([f(v) * C * 1000 for v in Vc_rk]) # mA

#  Plot: Vc comparison
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

ax = axes[0]
ax.plot(t_ex*1000, Vc_ex, 'b-', lw=2, label='Exact: $5(1-e^{-25t})$')
ax.plot(t_rk*1000, Vc_rk, 'r--', lw=1.5, label='RK4 (h=0.5 ms)')
```

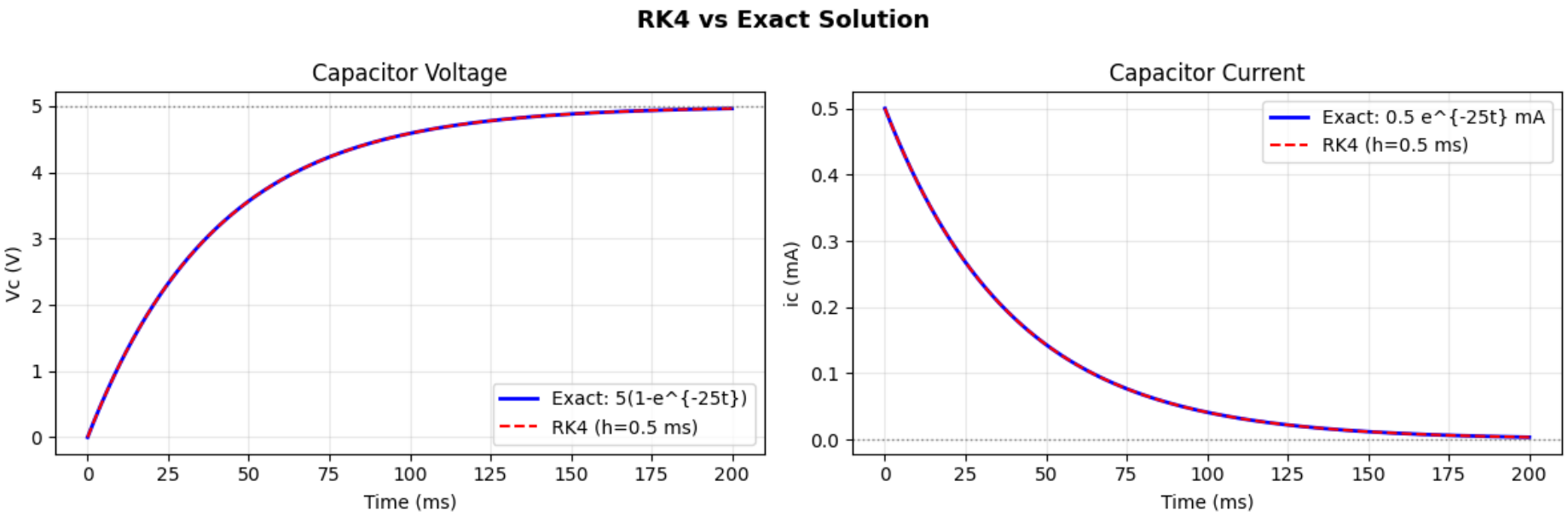
RK4 Verification – Python Code III

```
ax.axhline(5, color='gray', ls=':', lw=1)
ax.set_xlabel('Time (ms)')
ax.set_ylabel('$V_C$ (V)')
ax.set_title('Capacitor Voltage')
ax.legend()
ax.grid(True, alpha=0.3)

#  Plot: ic comparison
ax2 = axes[1]
ax2.plot(t_ex*1000, ic_ex, 'b-', lw=2, label='Exact: $0.5\,e^{-25t}$
    mA')
ax2.plot(t_rk*1000, ic_rk, 'r--', lw=1.5, label='RK4 (h=0.5 ms)')
ax2.axhline(0, color='gray', ls=':', lw=1)
ax2.set_xlabel('Time (ms)')
ax2.set_ylabel('$i_c$ (mA)')
ax2.set_title('Capacitor Current')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.suptitle('RK4 vs Exact Solution', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('figs/rk4.png', dpi=150)
```

RK4 Verification – Python Code IV

```
plt.show()

#  Max absolute error
Vc_ex_at_rk = Vinf * (1 - np.exp(-25 * t_rk))
err = np.max(np.abs(Vc_rk - Vc_ex_at_rk))
print(f"Max |error| in Vc: {err*1000:.4f} mV")
```

RK4 vs Exact Solution – Plot



RK4 result (dashed red) overlaps the exact theoretical curve (solid blue) confirming $i_c(t) = 0.50\, e^{-25t}\,\mathrm{mA}$. Max error $< 0.01\,\mathrm{mV}$ with $h = 0.5\,\mathrm{ms}$

NGSpice Netlist – rc_circuit.cir

```
* RC Circuit Transient Analysis  EC 2007 Problem 1.1.16
* Nodes: 1 = V_s positive terminal, 2 = capacitor node, 0 = GND

Vs 1 0 DC 10V ; 10 V DC voltage source

R1 1 2 20k ; Series resistor 20 kOhm
R2 2 0 20k ; Shunt resistor 20 kOhm
C1 2 0 4u IC=0 ; Capacitor 4 uF, initial voltage = 0 V

* Transient analysis: step=0.5ms, end=200ms, use initial conditions
.tran 0.5m 200m UIC

* Save node voltages and branch currents
.save v(2) i(C1)

* Print max voltage and current for quick check
.measure tran Vc_init FIND v(2) AT=0
.measure tran Vc_final FIND v(2) AT=200m
.measure tran ic_init FIND i(C1) AT=0

.end
```

Run with: `ngspice -b rc_circuit.cir -o out.raw`

NGSpice – Plot Results with Python I

```
import numpy as np
import matplotlib.pyplot as plt
import subprocess, os

#  1. Run NGSpice and capture output
spice_file = "rc_circuit.cir"
result = subprocess.run(
   ["ngspice", "-b", spice_file],
   capture_output=True, text=True
)

#  2. Parse NGSpice raw output (ASCII format)
# Alternatively, write .print tran v(2) i(C1) to a .dat file
# and load with np.loadtxt.
#
# For a portable approach, use the inline data from the
# .print statement by adding to the netlist:
# .print tran v(2) i(C1)
# and redirecting: ngspice -b rc_circuit.cir > spice_out.txt

data = np.loadtxt("spice_out.txt", comments=('#', 'Index'))
t_sp = data[:, 1] # time (s)
```

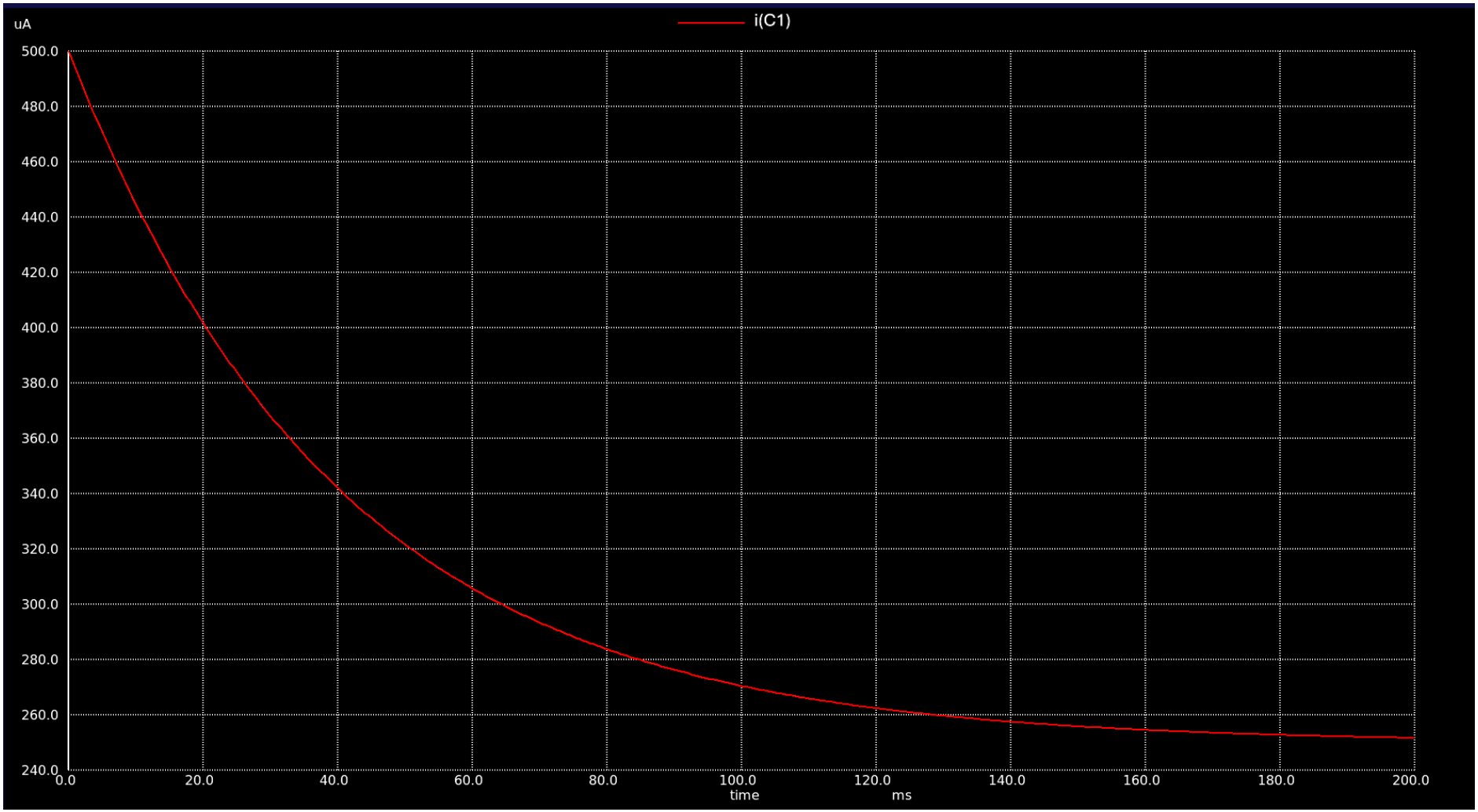
NGSpice – Plot Results with Python II

```
Vc_sp = data[:, 2] # v(2) (V)
ic_sp = data[:, 3] * 1000 # i(C1) converted to mA

#  3. Exact theoretical curves
t_ex = np.linspace(0, 0.2, 2000)
Vc_ex = 5 * (1 - np.exp(-25 * t_ex))
ic_ex = 0.5 * np.exp(-25 * t_ex) # mA

#  4. Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(t_ex*1000, Vc_ex, 'b-', lw=2, label='Exact theory')
ax1.plot(t_sp*1000, Vc_sp, 'r--', lw=1.5, label='NGSpice')
ax1.set_xlabel('Time (ms)'); ax1.set_ylabel('$V_C$ (V)')
ax1.set_title('Capacitor Voltage $V_C(t)$')
ax1.legend(); ax1.grid(True, alpha=0.3)

ax2.plot(t_ex*1000, ic_ex, 'b-', lw=2, label='Exact theory')
ax2.plot(t_sp*1000, ic_sp, 'r--', lw=1.5, label='NGSpice')
ax2.set_xlabel('Time (ms)'); ax2.set_ylabel('$i_c$ (mA)')
ax2.set_title('Capacitor Current $i_c(t)$')
ax2.legend(); ax2.grid(True, alpha=0.3)
```

NGSpice – Plot Results with Python III

```
plt.suptitle('NGSpice Simulation vs Exact Solution',
          fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('figs/spice.png', dpi=150)
plt.show()
```

NGSpice Simulation – Results



NGSpice transient simulation (dashed red) matches the exact solution (solid blue) confirming $i_c(0^+) = 0.50\,\mathrm{mA}$ and $V_C(\infty) = 5\,\mathrm{V}$.

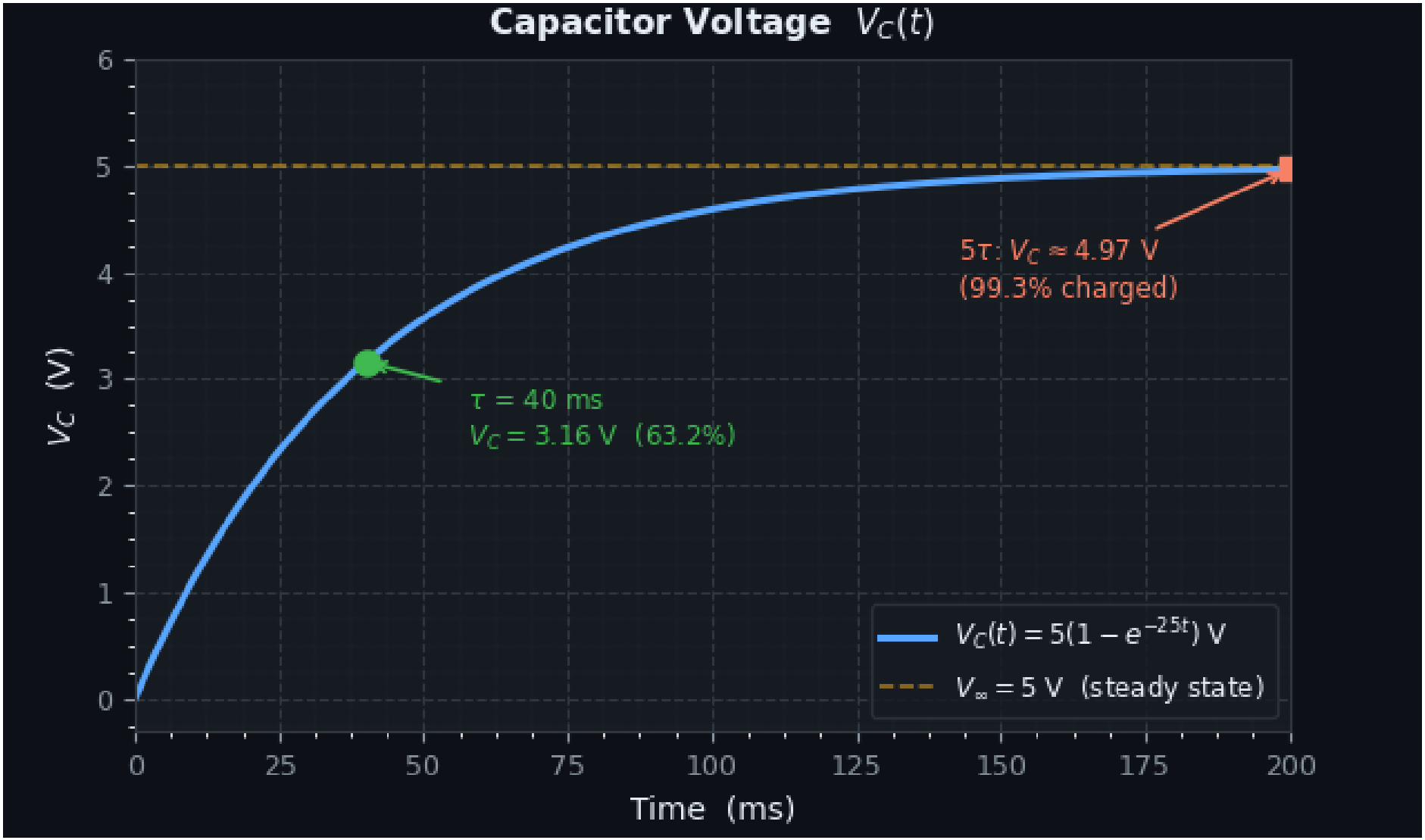
Capacitor Voltage – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 0.2, 1000)
V = 5
tau = 0.04

Vc = V*(1 - np.exp(-t/tau))

plt.figure(figsize=(8, 4))
plt.plot(t*1000, Vc, 'b-', lw=2, label='$V_C(t) = 5(1-e^{-25t})$ V')
plt.axhline(5, linestyle='--', color='gray', label='$V_\infty = 5$ V')
plt.xlabel('Time (ms)')
plt.ylabel('$V_C$ (V)')
plt.title('Capacitor Voltage $V_C(t)$')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('figs/Vc.png', dpi=150)
plt.show()
```

Capacitor Voltage – C Code I

```
#include <stdio.h>
#include <math.h>

int main(){
   double t, V=5, tau=0.04;

   for(int i=0;i<=200;i++){
      t = i/1000.0;
      double Vc = V*(1-exp(-t/tau));
      printf("%lf %lf\n", t*1000, Vc);
   }
   return 0;
}
```

Capacitor Voltage Plot



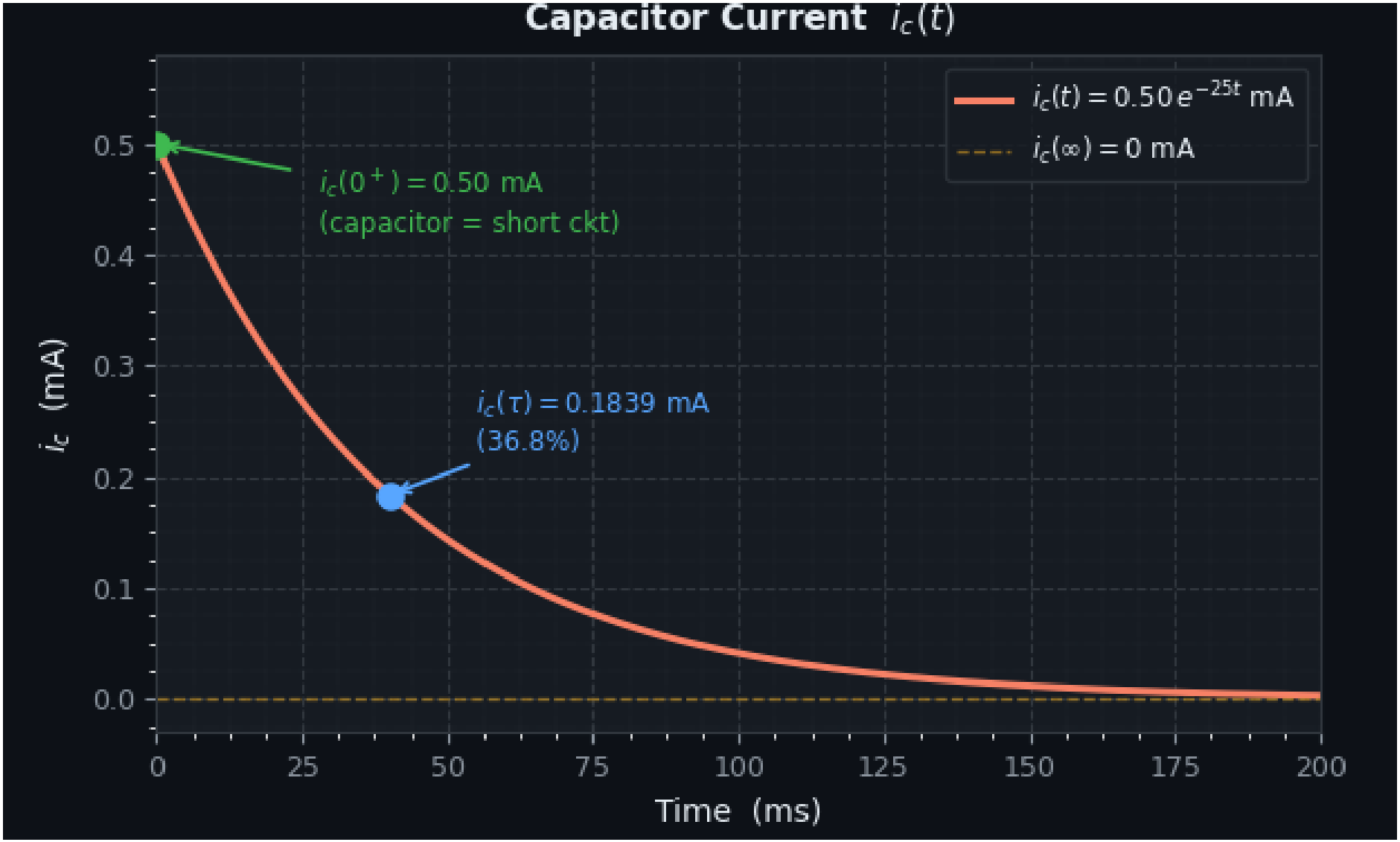
Capacitor Current – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 0.2, 1000)
V = 5
R = 10000
tau = 0.04

ic = (V/R)*np.exp(-t/tau)*1000

plt.figure(figsize=(8, 4))
plt.plot(t*1000, ic, 'r-', lw=2, label='$i_c(t) = 0.50\,e^{-25t}$ mA')
plt.axhline(0, linestyle='--', color='gray')
plt.xlabel('Time (ms)')
plt.ylabel('$i_c$ (mA)')
plt.title('Capacitor Current $i_c(t)$')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('figs/Ic.png', dpi=150)
plt.show()
```

Capacitor Current – C Code I

```
#include <stdio.h>
#include <math.h>

int main(){
   double t, V=5, R=10000, tau=0.04;

   for(int i=0;i<=200;i++){
      t = i/1000.0;
      double ic = (V/R)*exp(-t/tau)*1000;
      printf("%lf %lf\n", t*1000, ic);
   }
   return 0;
}
```

Capacitor Current Plot



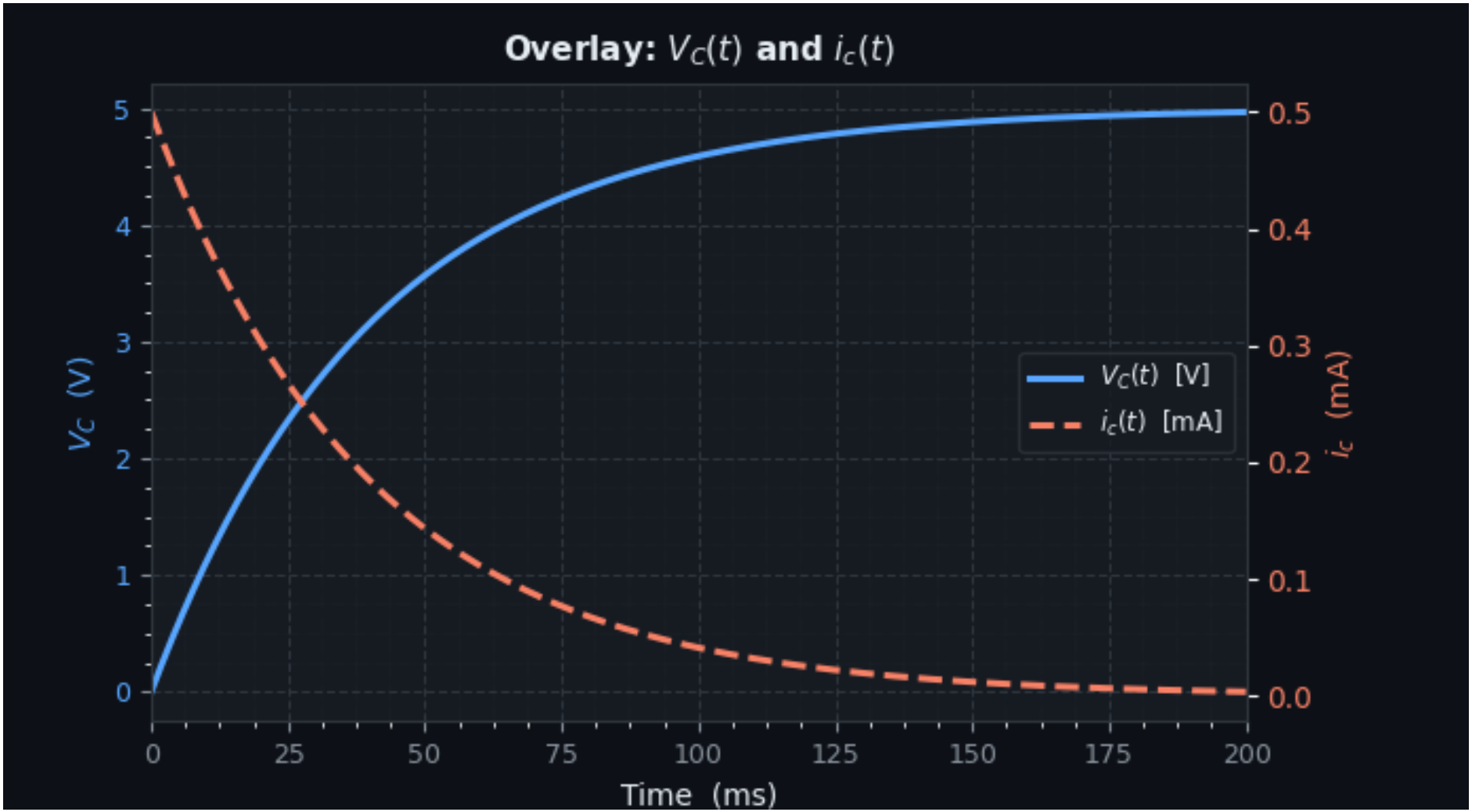
Overlay Plot – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 0.2, 1000)
V = 5
tau = 0.04
Vc = V * (1 - np.exp(-t/tau))
ic = (V/10000) * np.exp(-t/tau) * 1000

fig, ax1 = plt.subplots(figsize=(8, 4))
ax1.plot(t*1000, Vc, 'b-', lw=2, label='$V_C(t)$ (V)')
ax1.set_xlabel("Time (ms)")
ax1.set_ylabel("$V_C$ (V)", color='blue')
ax2 = ax1.twinx()
ax2.plot(t*1000, ic, 'r--', lw=2, label='$i_c(t)$ (mA)')
ax2.set_ylabel("$i_c$ (mA)", color='red')

fig.legend(loc='center right', bbox_to_anchor=(0.88, 0.5))
plt.title('Overlay: $V_C(t)$ and $i_c(t)$')
plt.savefig('figs/O.png', dpi=150)
plt.show()
```

Overlay Plot – C Code I

```
#include <stdio.h>
#include <math.h>

int main() {
   double t, V=5, tau=0.04;

   for(int i=0;i<=200;i++){
      t = i/1000.0;
      double Vc = V*(1-exp(-t/tau));
      double ic = (V/10000.0)*exp(-t/tau)*1000;
      printf("%lf %lf %lf\n", t*1000, Vc, ic);
   }
   return 0;
}
```

Overlay Plot



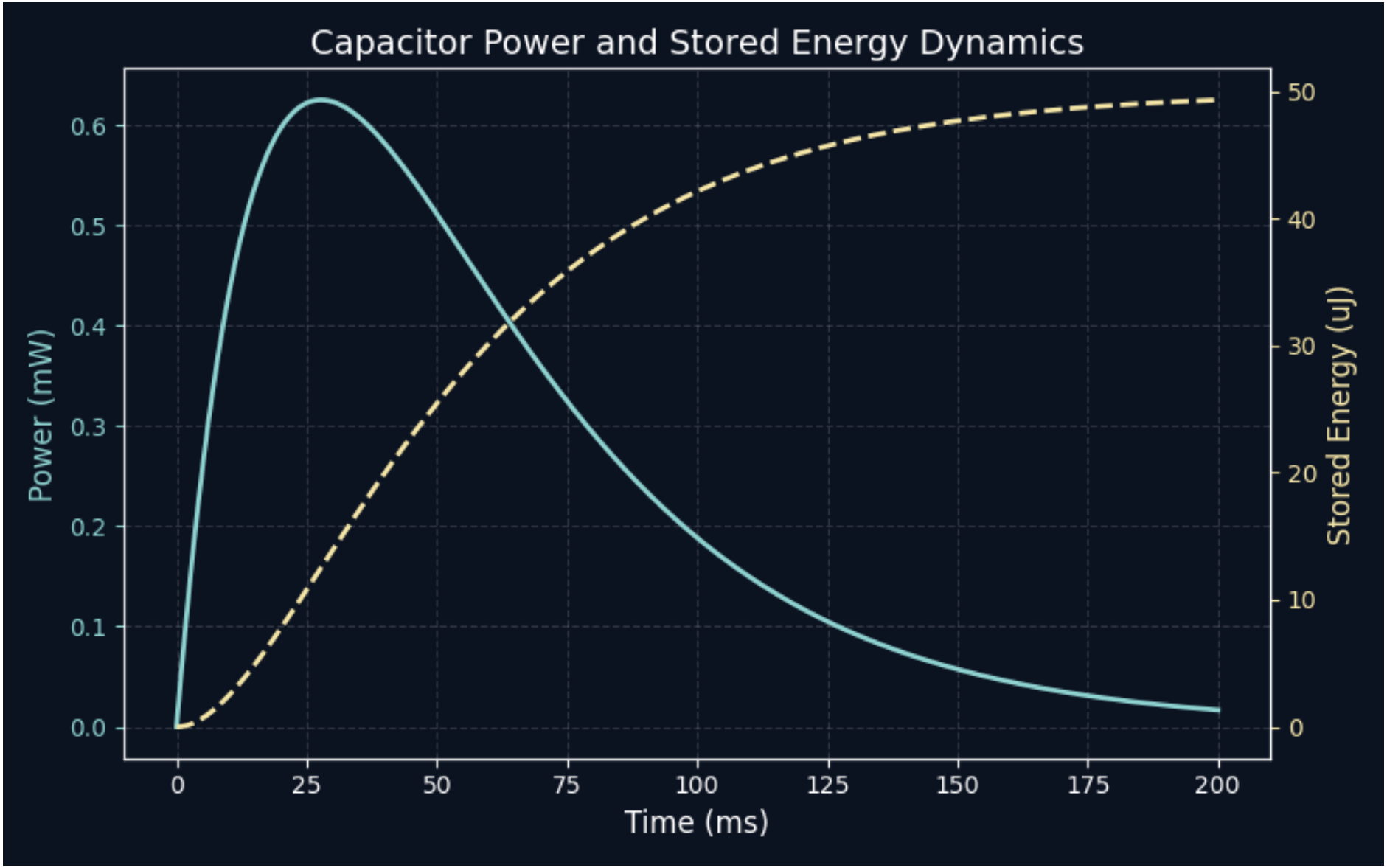
Power & Energy – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 0.2, 1000)
V = 5
tau = 0.04
C = 4e-6

Vc = V * (1 - np.exp(-t/tau))
ic = (V / 10000) * np.exp(-t/tau)

power_mW = (Vc * ic) * 1000
energy_uJ = (0.5 * C * Vc**2) * 1e6

bg = '#0b1220'
power_color = '#8fd3d1'
energy_color = '#f5e6a8'

fig, ax1 = plt.subplots(figsize=(9, 4))
fig.patch.set_facecolor(bg)
ax1.set_facecolor(bg)
```

Power & Energy – Python Code II

```
ax1.plot(t*1000, power_mW, color=power_color, lw=2, label='Power (mW)
    ')
ax1.set_xlabel("Time (ms)", color='white')
ax1.set_ylabel("Power (mW)", color=power_color)
ax1.tick_params(colors='white')

ax2 = ax1.twinx()
ax2.plot(t*1000, energy_uJ, color=energy_color, lw=2,
       linestyle='--', label='Energy (uJ)')
ax2.set_ylabel("Stored Energy ($\\mu$J)", color=energy_color)
ax2.tick_params(colors='white')

plt.title('Capacitor Power and Stored Energy Dynamics',
        color='white')
plt.savefig('figs/PS.png', dpi=150, facecolor=bg)
plt.show()
```

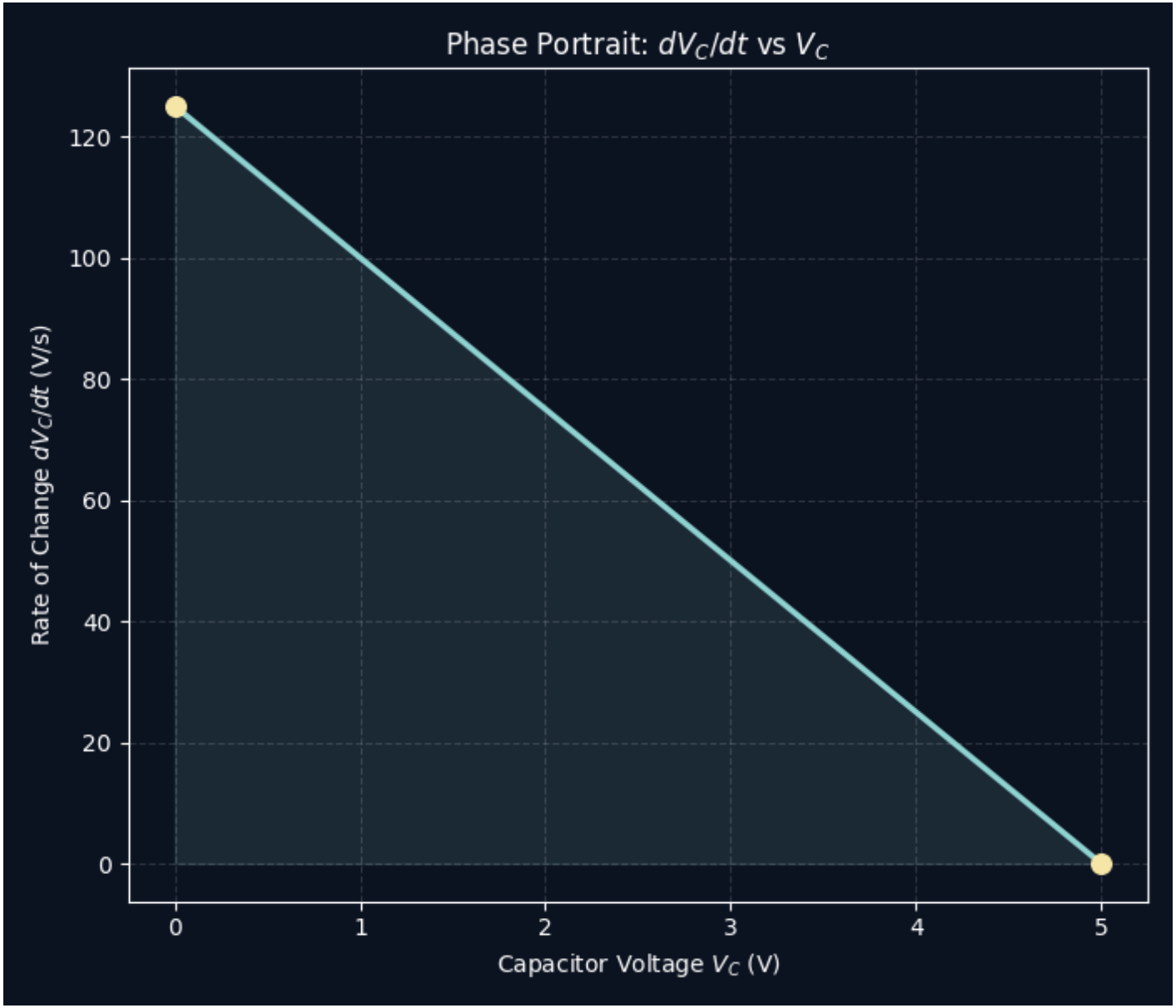
Plot – Power & Energy



Phase Portrait – Python Code I

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 0.2, 1000)
V = 5
tau = 0.04

Vc = V * (1 - np.exp(-t/tau))
dVc_dt = (V / tau) * np.exp(-t/tau)

bg = '#0b1220'
curve = '#8fd3d1'
accent = '#f5e6a8'

fig, ax = plt.subplots(figsize=(7, 5))
fig.patch.set_facecolor(bg)
ax.set_facecolor(bg)
ax.plot(Vc, dVc_dt, color=curve, lw=2)
ax.fill_between(Vc, dVc_dt, alpha=0.12, color=curve)
ax.scatter([0], [125], color=accent, zorder=5, s=60,
        label='Initial ($V_C=0$, $dV/dt=125$)')
ax.scatter([5], [0], color=accent, zorder=5, s=60,
```

Phase Portrait – Python Code II

```
        label='Equilibrium ($V_C=5$ V)')
ax.set_xlabel('Capacitor Voltage $V_C$ (V)', color='white')
ax.set_ylabel('Rate of Change $dV_C/dt$ (V/s)', color='white')
ax.set_title('Phase Portrait: $dV_C/dt$ vs $V_C$', color='white')
ax.tick_params(colors='white')
ax.legend(facecolor=bg, labelcolor='white')
ax.grid(True, alpha=0.2)
plt.savefig('figs/PP.png', dpi=150, facecolor=bg)
plt.show()
```

Phase Portrait